

Chatbot Académico de Recuperación basada en Corpus

Atención Automatizada de Consultas Universitarias en la EPIIS-UNSAAC

Castro Pari, Rayneld Fidel · Florez Vega, Yamir Wagner · Mamani Flores, Natan · Merma Ccarhuarupay, Adel Alejandro

Escuela Profesional de Ingeniería Informática y de Sistemas — Facultad de Ingeniería Eléctrica, Electrónica, Informática y Mecánica

IF651 – Inteligencia Artificial · Semestre 2026-I

Recorrido de la exposición



01

Problema y
enfoque



02

Arquitectura
del sistema



03

Procesamiento
del lenguaje



04

Calibración y
comportamiento



05

Resultados y
conclusiones

Consultas frecuentes en un dominio cerrado y reglamentado

Áreas de consulta recurrente en la EPIIS

Tutorías	Cursos	Especialidades
Prácticas pre-profesionales	Bienestar	Movilidad
Matrícula	Titulación	Servicios universitarios

Requisitos del sistema

-  **Trazabilidad**
saber por qué se eligió cada respuesta
-  **Mantenibilidad**
actualizar JSON sin tocar el motor
-  **Rapidez**
búsquedas en memoria, costo $O(1)$
-  **Corrección lingüística**
errores comunes y variaciones

El sistema recupera una respuesta institucional ya validada; ningún componente genera contenido nuevo a partir de un modelo entrenado.

Reglas, recuperación o un modelo de lenguaje (LLM)

Criterio	Reglas	Recuperación	LLM
Respuesta controlada	Alta	Alta	Variable
Trazabilidad	Alta	Alta	Dependiente del diseño
Cobertura lingüística	Baja	Media	Alta
Costo de ejecución	Bajo	Bajo	Medio / alto
Dependencia externa	No	No	Frecuente
Riesgo de alucinación	Nulo	Nulo	Presente



Se prioriza una respuesta determinista, verificable y siempre asociada a una fuente del corpus.

Cinco capas lógicas, separación física parcial



Presentación

HTML / CSS / ipywidgets del cuaderno — integrada al cuaderno



Aplicación y orquestación

ChatbotEPIIS, ConversationHistory — chatbot_epiis.py aún vacío



Procesamiento y decisión

Preprocessor, IntentMatcher — versión modular más simple que el cuaderno



Acceso y respuesta

KnowledgeBaseLoader, ResponseBuilder — cargador modular con error en faq_index



Persistencia y soporte

corpus/, knowledge_base/, knowledge_files/, sources/, tools/

Seis clases; un patrón Facade explícito



Preprocessor

Normaliza, elimina tildes y signos, filtra stopwords, tokeniza y genera raíces.



KnowledgeBaseLoader

Carga los siete JSON y construye diccionarios por id e intención.



IntentMatcher

Calcula IDF, corrige errores, puntúa intenciones, aplica contexto.



ResponseBuilder

Recupera la entrada del corpus y arma la respuesta estructurada.



ConversationHistory

Registra turnos, scores, categorías y métricas de resolución.



ChatbotEPIIS (Facade)

Conecta todos los componentes y expone la operación respond().

El patrón Strategy aún no está formalmente implementado: se describe como una posible evolución, no como característica consolidada.

Flujo de decisión de una consulta



Normalización, IDF y coincidencia difusa

Normalización (Preprocessor)



Salida: (t_norm, T, T_exp) — texto normalizado, tokens y conjunto expandido

Pesos y coincidencia

w_primaria	3.0
w_secundaria	1.5
w_negativa	2.0
bono frase exacta	2.5

Fuzzy matching: SequenceMatcher, umbral 0.78
Contexto conversacional: × 1.20 a la categoría previa

Ponderación IDF

$$IDF(t) = \log \left(\frac{N+1}{df(t)+1} \right) + 1$$

N = 100 intenciones

647

términos IDF

618

raíces

631

entradas difusas

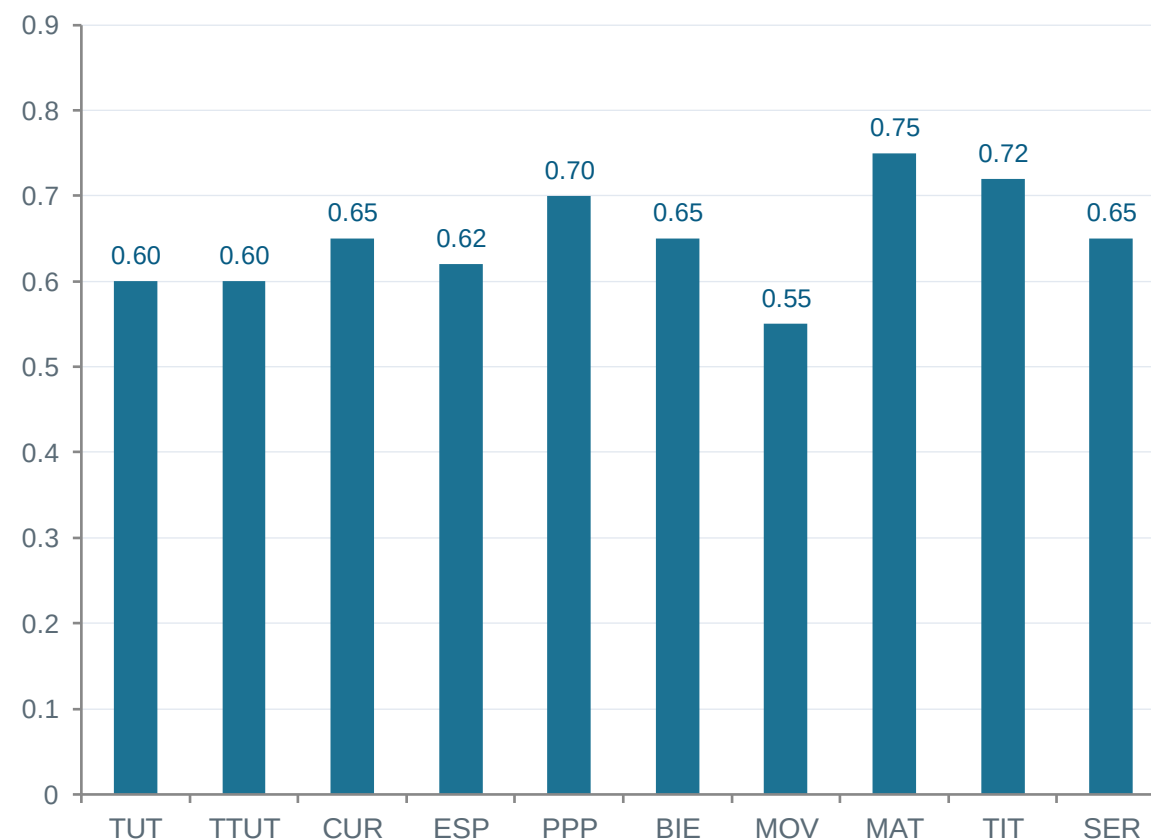
La ejecución limpia del cuaderno construye estos índices a partir del corpus actual de 100 intenciones.

Umbrales por categoría (confianza_minima)

$$\theta_i = \max(p_c(i), 0.50, \min(0.95, 0.90 \times s_{i_min}))$$

- 4 consultas de validación por intención
- margen de seguridad del 10 % hacia abajo
- piso absoluto 0.50 · techo 0.95
- piso específico por categoría (p_c)

En la práctica, los cien resultados finales quedaron determinados por el piso de su categoría: piso_categoria_aplicado en el 100 % de las intenciones.



Por qué el score puede superar 1.0



Frase disparadora literal

Devuelve directamente un score de 1.0



Bonificación de especificidad

$0.75 + 0.07 \times \min(n, 4) \rightarrow$ hasta 1.03



Refuerzo contextual

$\text{CONTEXT_BOOST} \times 1.20$ sobre la última categoría

Ejemplo: un puntaje de 0.85 puede convertirse en 1.02 tras el refuerzo contextual.

Corrección visual recomendada

```
pct = min(int(resp["score"] * 100), 100)
```

Solo limita la representación visual de la barra; no altera el ranking interno ni la comparación con los umbrales.



Debe llamarse “puntaje de coincidencia”, no “confianza” ni “probabilidad”.

Corpus de 100 entradas y el índice `faq_index`

100

preguntas

10

categorías

300variaciones
lingüísticas

10 categorías × 10 entradas × 3 variaciones por entrada.

`faq_index.json`

NO almacena respuestas ni clasifica lenguaje. Es un índice auxiliar con 5 campos: versión, total de preguntas, total de categorías, resumen índice y `busqueda_rapida`.

Mapea: `intent_id` → `id_del_corpus`

Ejemplo de mapeo

`consulta_tutoria_definicion`**TUT-001**

Búsqueda por diccionario — costo promedio $O(1)$



El módulo `knowledge_base_loader.py` todavía busca una subclave “`mapa_muestra`” inexistente: debe corregirse antes de equiparlo al cuaderno.

ConversationHistory: registra, no decide

Registra por cada turno

- Fecha y consulta original
- Texto normalizado
- Intención, score y categoría
- Indicador de fallback y respuesta construida

Calcula a partir del historial

- Número de turnos
- Tasa de resolución y de fallback
- Distribución por categoría · reporte textual



No modifica las respuestas posteriores

Ningún dato de `_turnos` se envía a `IntentMatcher` ni a `ResponseBuilder`. No existe aprendizaje en línea.

La única adaptación contextual real:

`IntentMatcher._last_category`

Refuerza $\times 1.20$ a las candidatas de la categoría previa, solo en el siguiente turno.

Por qué no se incorporó un LLM



**Dominio institucional finito y
reglamentado**



**Trazabilidad directa de la fuente
normativa**



**Comportamiento determinista y
reproducible**



**Sin claves de API ni
infraestructura de inferencia**



Menor costo y latencia variable



**No importa transformers ni
openai — solo Python estándar**

Un LLM podría incorporarse luego como reformulador o clasificador semántico, pero la respuesta final debe seguir anclada al corpus y a sus fuentes.

Ejecución limpia: 29 de 41 casos correctos

70.7%

29 de 41 casos correctos

6 intención incorrecta

6 rechazo por fallback

La salida 34/41 guardada en el cuaderno corresponde a una versión anterior, sin el umbral por intención activo.

Fallos representativos

Consulta	Resultado
Jalar un curso	Fallback (0.581 < 0.65)
Cuándo elegir especialidad	Fallback (0.532 < 0.62)
Duración de titulación	Fallback (0.383 < 0.72)
Consulta genérica sobre PPP	Fallback (0.265 < 0.70)
Proceso de matrícula “Serunsa”	Fallback — keyword ausente

Limitaciones y conclusiones

Limitaciones

- Consultas de prueba redactadas por el propio equipo
- Sin conjunto independiente fuera de dominio
- El calibrador no compite contra las otras 99 intenciones
- Los pisos por categoría dominan los 100 resultados
- El puntaje no está calibrado como probabilidad
- Cuaderno, exportación y módulos aún no sincronizados



Conclusión

El proyecto constituye un prototipo institucional trazable y de bajo costo.

Aún requiere sincronizar las versiones de cuaderno, exportación y módulos, y ampliar la evaluación con consultas reales y fuera de dominio.

¿Preguntas?